In [ ]:
```
# Initialize Otter
import otter
grader = otter.Notebook("practice08.ipynb")
```

Student names and e-mails:

YOUR NAME — your@calvin.edu

YOUR NAME — your@calvin.edu

Practice 08 — Miles per Gallon: Linear Regression, Ridge and Lasso

In 1973, an oil embargo quadrupled the price of oil, and American drivers waited in long lines at gas stations. Two years later, Congress set fuel-economy standards for new cars, starting with the 1978 models. Suddenly, how far a car could go on a gallon of gas mattered.

Can a line predict a car's fuel economy from its weight, its engine and its year? And what goes wrong when it learns from only a handful of cars?

SLO	Description
10A	Train and interpret linear regression models
10B	Apply Ridge and Lasso regularization to reduce overfitting and explain the effect of the regularization parameter
10C	Compute and interpret regression performance metrics — MAE, RMSE, and R^2

The Dataset: 398 Cars, 1970–1982

These are real cars, from the StatLib library at Carnegie Mellon University. The data was used at the 1983 American Statistical Association Exposition, and it now lives in the UCI Machine Learning Repository ("Auto MPG", CC BY 4.0). Its documentation says: *"The data concerns city-cycle fuel consumption in miles per gallon."*

Column	What it holds
`name`	make and model, e.g. "ford pinto"
`origin`	where the car was made: USA, Europe or Japan
`model_year`	1970 to 1982
`cylinders`	number of engine cylinders, 3 to 8
`displacement`	engine size, in cubic inches
`horsepower`	engine power (6 cars have a blank)
`weight`	in pounds (lb)
`acceleration`	time to accelerate, in seconds: a bigger number is a slower car
`mpg`	**the label**: miles per gallon

Setup (given — just run it)

In [ ]:
```
import numpy as np
import pandas as pd
import plotly.express as px
from sklearn.model_selection import train_test_split, cross_validate, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score

cars = pd.read_csv("https://cs.calvin.edu/courses/data/202/26fa/datasets/mpg.csv")
print(len(cars), "cars | average:", round(cars["mpg"].mean(), 1), "mpg")
cars.head()
```

The features are the six number columns and `origin`. The split is given: 25% of the cars for testing.

In [ ]:
```
numeric = ["cylinders", "displacement", "horsepower", "weight", "acceleration", "model_year"]
X = cars[numeric + ["origin"]]
y = cars["mpg"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=1)
print(len(X_train), "training cars |", len(X_test), "test cars")
```

Part 1 — One Line (SLO 10A)

In [ ]:
```
fig = px.scatter(x=X_train["weight"], y=y_train, opacity=0.5, labels={"x": "weight (lb)", "y": "miles per gallon"},
                 title="298 Training Cars: Fuel Economy vs. Weight")
fig.show()
```

> 🎯 **Predict First:** Draw a straight line through the cloud in your head. About how many miles per gallon does an extra 1,000 lb cost?

Task 10A.1 — One Line Through Weight *(2 pts)*

1. Fit a `LinearRegression` on `X_train[["weight"]]` and `y_train`. Store it in `line`.
2. Store its slope in `slope` and its intercept in `intercept`. (`coef_` holds one slope per column, so take its first element.)
3. Store the predicted mpg of a 3,000 lb car in `mpg_3000`. (*Hint:* `predict` needs a table, `pd.DataFrame({"weight": [3000]})`, and returns an array.)

In [ ]:
```
line = LinearRegression().fit(X_train[["weight"]], y_train)  # SOLUTION
slope = line.coef_[0]  # SOLUTION
intercept = line.intercept_  # SOLUTION
mpg_3000 = line.predict(pd.DataFrame({"weight": [3000]}))[0]  # SOLUTION

print(f"mpg = {intercept:.2f} + ({slope:.5f}) × weight")
print(f"1,000 lb more: {slope * 1000:+.1f} mpg | a 3,000 lb car: {mpg_3000:.1f} mpg")
```

In [ ]:
```
grader.check("10A.1")
```

Each extra 1,000 lb goes with about **7.8 fewer** miles per gallon. The intercept, 46.6 mpg for a car that weighs nothing, only positions the line: no car weighs 0 lb.

Part 2 — How Far Off? (SLO 10C)

As in Monday's class, score the line on the **test** cars.

Task 10C.1 — MAE, RMSE and R² *(2 pts)*

1. Store the line's predictions for the test cars, `X_test[["weight"]]`, in `predicted`.
2. Store the line's MAE, RMSE and R² on the test cars in `mae`, `rmse` and `r2`. The three functions were imported in the setup; each takes the actual values first, then the predictions.

The lazy model is given.

In [ ]:
```
predicted = line.predict(X_test[["weight"]])  # SOLUTION
mae = mean_absolute_error(y_test, predicted)  # SOLUTION
rmse = root_mean_squared_error(y_test, predicted)  # SOLUTION
r2 = r2_score(y_test, predicted)  # SOLUTION

lazy = np.full(len(y_test), y_train.mean())          # always guess the training average
print(f"line | MAE {mae:.2f} | RMSE {rmse:.2f} | R² {r2:.2f}")
print(f"lazy | MAE {mean_absolute_error(y_test, lazy):.2f} | RMSE {root_mean_squared_err
or(y_test, lazy):.2f} | R² {r2_score(y_test, lazy):.2f}")
```

In [ ]:
```
grader.check("10C.1")
```

The line is typically about **2.9 mpg** off (MAE). RMSE, 3.8, counts the big misses more. $R^2 = 0.74$: the line removes about three quarters of the lazy model's squared error.

> 📌 **One split:** these are 100 test cars.

Task 10C.2 — Who Does the Line Get Wrong? *(2 pts)*

`era` (given) groups the test cars by model year.

1. Store each test car's residual, actual − predicted, in `residuals`.
2. Store the average residual for each era in `by_era`. (*Hint:* `groupby(era)`, then `mean()`.)

In [ ]:
```
era = pd.cut(X_test["model_year"], [1969, 1973, 1977, 1982], labels=["1970-73", "1974-77", "1978-82"])

residuals = y_test - predicted  # SOLUTION
by_era = residuals.groupby(era).mean()  # SOLUTION

print(by_era.round(2))
```

In [ ]:
```
grader.check("10C.2")
```

A positive residual means a car gets **more** miles per gallon than its weight predicts. The 1970–73 cars average 2.3 mpg *below* the line, and the 1978–82 cars 2.3 mpg *above* it. After the oil crisis and the new standards, carmakers got more miles out of every pound. A line on weight alone can't know a car's year.

Part 3 — More Columns (SLO 10A)

Task 10A.2 — Weight and Year Together *(2 pts)*

Fit a `LinearRegression` on `X_train[["weight", "model_year"]]` and `y_train`. Store it in `two`. Then store its weight slope in `weight_slope` and its model-year slope in `year_slope` (in the same order as the columns).

In [ ]:
```
two = LinearRegression().fit(X_train[["weight", "model_year"]], y_train)  # SOLUTION
weight_slope = two.coef_[0]  # SOLUTION
year_slope = two.coef_[1]  # SOLUTION

print(f"1,000 lb more, same model year: {weight_slope * 1000:+.1f} mpg (weight alone: {slope * 1000:+.1f})")
print(f"one model year newer, same weight: {year_slope:+.2f} mpg")
```

In [ ]:
```
grader.check("10A.2")
```

Each slope now means "**holding the other column fixed**." At the same weight, each newer model year goes with 0.8 more mpg. At the same model year, 1,000 lb cost 6.7 mpg instead of 7.8: alone, weight also took the blame for the older cars, which were heavier *and* less efficient.

All Columns, in a Pipeline (given — just run it)

The six numbers (blank horsepower filled with the median, as in Week 9, then scaled), plus `origin`, one-hot with `drop="first"` so that Europe becomes the baseline:

In [ ]:
```
model = Pipeline([
    ("prep", ColumnTransformer([
        ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), numer
ic),
        ("words", OneHotEncoder(drop="first"), ["origin"]),
    ])),
    ("line", LinearRegression()),
])
scores = cross_validate(model, X_train, y_train, cv=5, scoring="neg_root_mean_squared_er
ror", return_train_score=True)
print(f"all columns | RMSE on the folds it learned from: {-scores['train_score'].mean
():.2f} | on the held-out fold: {-scores['test_score'].mean():.2f}")
```

(scikit-learn reports *negative* RMSE, so the minus signs turn it back.) A small gap, 3.39 against 3.53: with 298 cars, 8 coefficients don't overfit.

Part 4 — A Small Garage: Overfitting and Penalties (SLO 10B)

Suppose a magazine had road-tested only **10** cars. To keep things simple, this part uses the six number columns only.

In [ ]:
```
few = X_train.sample(10, random_state=1)[numeric]        # pretend these 10 are all the
cars we have
few_mpg = y_train[few.index]
others = X_train.drop(few.index)[numeric]                # the other 288 training cars s
tand in for new cars
others_mpg = y_train[others.index]
display(cars.loc[few.index, ["name", "model_year", "weight", "displacement", "cylinder
s", "horsepower", "mpg"]])

small = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), LinearRegressi
on()).fit(few, few_mpg)
everything = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), LinearReg
ression()).fit(X_train[numeric], y_train)
print(f"line from 10 cars | RMSE on its 10 cars: {root_mean_squared_error(few_mpg, smal
l.predict(few)):.2f}"
      f" | on the other 288: {root_mean_squared_error(others_mpg, small.predict(other
s)):.2f}")
print("other cars predicted below 0 mpg:", (small.predict(others) < 0).sum())
```

> 🎯 **Predict First:** Below are the line's slopes, in mpg per standard deviation of each column, learned from the 10 cars and from all 298. Which ones look **too big**? Which point the **wrong way**?

In [ ]:
```
# small[-1] is the pipeline's last step, the LinearRegression
display(pd.DataFrame({"from all 298 cars": everything[-1].coef_, "from 10 cars": small[-1].coef_}, index=numeric).round(2))
```

Three symptoms of overfitting in a line:

- **Too big:** displacement −30.2 mpg per typical step, on cars that average 23.6 mpg
- **The wrong way:** weight **+15.0** and cylinders +10.9. Heavier cars, with more cylinders, get *more* miles per gallon?
- **A tug of war:** in these 10 cars, weight, displacement, cylinders and horsepower are near-copies (correlations from 0.91 to 0.99). The line pushes hard on some and pulls hard on others. The pushes cancel on these 10 cars, not on new ones: 19 of the other 288 cars are predicted to get *less than zero* miles per gallon

Task 10B.1 — Ridge: Choose alpha *(2 pts)*

Build a `GridSearchCV` called `ridge_search` on the pipeline `make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), Ridge())`, with the grid `{"ridge__alpha": alphas}`, `cv=5`, `scoring="neg_root_mean_squared_error"` and `return_train_score=True`. Fit it on the 10 cars: `few` and `few_mpg`.

(`ridge__alpha` is the `alpha` of the step named `ridge`: `make_pipeline` names each step after its class.) The plot is given.

In [ ]:
```
alphas = [0.01, 0.1, 1, 10, 100, 1000]
ridge_search = GridSearchCV(make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), Ridge()),  # SOLUTION
# BEGIN SOLUTION NO PROMPT
                            {"ridge__alpha": alphas}, cv=5, scoring="neg_root_mean_squared_error",
                            return_train_score=True)
# END SOLUTION
ridge_search.fit(few, few_mpg)  # SOLUTION

results = pd.DataFrame({"alpha": alphas,
                        "folds it learned from": -ridge_search.cv_results_["mean_train_score"],
                        "held-out fold": -ridge_search.cv_results_["mean_test_score"]})
px.line(results, x="alpha", y=["folds it learned from", "held-out fold"], markers=True, log_x=True,
        labels={"value": "RMSE (mpg)", "variable": "scored on"}, title="Ridge on 10 Cars: Choosing alpha by 5-Fold CV").show()
print("best alpha:", ridge_search.best_params_["ridge__alpha"], "| held-out RMSE:", round(-ridge_search.best_score_, 2))
```

In [ ]:
```
grader.check("10B.1")
```

Read the plot from left to right: with almost no penalty, the line misses the folds it learned from by 1 to 2 mpg and the held-out fold by more than 6: **overfitting**. At alpha = 1,000, both errors are high: **underfitting**. The best alpha is in between. Here a **bigger** alpha means a **simpler** model.

> 📌 **One sample:** each held-out fold is only 2 cars, so this curve is shaky.

What did Ridge do to the slopes? (given)

In [ ]:
```
display(pd.DataFrame({"plain line, 10 cars": small[-1].coef_,
                      "Ridge, 10 cars": ridge_search.best_estimator_[-1].coef_}, index=n
umeric).round(2))
```

Ridge **shares the blame**: weight, displacement, cylinders and horsepower each get a small slope, about −1.2 to −1.4. No more tug of war, and every sign points the sensible way.

Task 10B.2 — Lasso: Which Columns Survive? *(2 pts)*

Do the same with Lasso: build a `GridSearchCV` called `lasso_search` on `make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), Lasso(max_iter=100_000))`, with the grid `{"lasso__alpha": lasso_alphas}`, `cv=5` and `scoring="neg_root_mean_squared_error"`, and fit it on the 10 cars. (`max_iter` lets Lasso's step-by-step search run long enough to settle.)

The kept and dropped columns are given.

In [ ]:
```
lasso_alphas = [0.01, 0.1, 0.3, 1, 3, 10]
lasso_search = GridSearchCV(make_pipeline(SimpleImputer(strategy="median"), StandardScal
er(), Lasso(max_iter=100_000)),  # SOLUTION
# BEGIN SOLUTION NO PROMPT
                            {"lasso__alpha": lasso_alphas}, cv=5, scoring="neg_root_mean
_squared_error")
# END SOLUTION
lasso_search.fit(few, few_mpg)  # SOLUTION

lasso_slopes = pd.Series(lasso_search.best_estimator_[-1].coef_, index=numeric)
print("best alpha:", lasso_search.best_params_["lasso__alpha"], "| held-out RMSE:", roun
d(-lasso_search.best_score_, 2))
print("kept:   ", list(lasso_slopes[lasso_slopes != 0].index))
print("dropped:", list(lasso_slopes[lasso_slopes == 0].index))
```

In [ ]:
```
grader.check("10B.2")
```

Lasso keeps **one** of the four near-copies, displacement, plus the model year, and drops the rest. Ridge shared the slope among the near-copies; Lasso picks one. (With another 10 cars, it could just as well have picked weight.)

Elastic Net: Both at Once (given — just run it)

Elastic net charges both penalties, mixed, with two knobs: `alpha` sets **how strong** the penalty is, and `l1_ratio` sets **the balance**: 1 is pure Lasso, 0 is pure Ridge. With two knobs, `GridSearchCV` tries every pair.

In [ ]:
```
net_search = GridSearchCV(make_pipeline(SimpleImputer(strategy="median"), StandardScaler
(), ElasticNet(max_iter=100_000)),
                          {"elasticnet__alpha": [0.01, 0.1, 0.3, 1, 3], "elasticnet__l1_
ratio": [0.1, 0.5, 0.9, 1]},
                          cv=5, scoring="neg_root_mean_squared_error")
net_search.fit(few, few_mpg)
print(net_search.best_params_, "| held-out RMSE:", round(-net_search.best_score_, 2))
```

On these 10 cars, cross-validation picks a mostly-Ridge balance (`l1_ratio = 0.1`).

The Final Comparison (given — just run it)

The other 288 training cars were never used to fit or tune these models, so they can stand in for new cars:

In [ ]:
```
one_line = LinearRegression().fit(few[["weight"]], few_mpg)
lazy_10 = np.full(len(others_mpg), few_mpg.mean())
display(pd.Series({"lazy: the 10 cars' average": root_mean_squared_error(others_mpg, laz
y_10),
                   "one line (weight)": root_mean_squared_error(others_mpg, one_line.pre
dict(others[["weight"]])),
                   "plain line, 6 columns": root_mean_squared_error(others_mpg, small.pr
edict(others)),
                   "Ridge": root_mean_squared_error(others_mpg, ridge_search.predict(oth
ers)),
                   "Lasso": root_mean_squared_error(others_mpg, lasso_search.predict(oth
ers)),
                   "elastic net": root_mean_squared_error(others_mpg, net_search.predict
(others))},
                  name="learned from 10 cars: RMSE on the other 288 (mpg)").round(2))
```

The plain line, with six columns for 10 cars, does worse than just guessing the 10 cars' average. The penalties bring it down to about 4 mpg, a little better than one line through weight.

Now, **once**, open the test set:

In [ ]:
```
print("test RMSE | plain line, all 298 training cars:", round(root_mean_squared_error(y_
test, everything.predict(X_test[numeric])), 2))
print("test RMSE | Ridge, only 10 cars:              ", round(root_mean_squared_error(y_
test, ridge_search.predict(X_test[numeric])), 2))
```

More data is the best cure: with 298 cars, the plain line needs no penalty. But with only 10, a penalty got surprisingly close.

> 📌 **One split:** these are 100 test cars, and one random set of 10.

Submission

Save your notebook, then upload the **`.ipynb` file** directly to our Moodle assignment page. Don't export, zip, or submit anything else — just the notebook.